Kosarak dataset

Load database

In [1]:
```
#STEP 1: LOAD DATABASE
import time
import pathlib
import Func_load_transaction_database as Func_load_DB
import psutil
import os
from datetime import datetime

now = datetime.now()
print("The last running =", now)

process = psutil.Process(os.getpid())

def memory_usage_mb():
    return process.memory_info().rss / 1024 / 1024  # MB

dbName = "Kosarak"
start_time = time.time()
DBtrans = Func_load_DB.Func_load_database(dbName)
print("Time to load database from .dat: %s seconds ---" % (time.tim
print('Load Transaction database successfully')
# print(DBtrans)
print(f"RAM usage after loading DB: {memory_usage_mb():.2f} MB")

max_level = 0 # Maximum level of itemsets to mine

# Robust thresholds for TOI mining
Sup = 0
Occ = 0
Pro = 0
# decreasing thresholds for higher levels
# (these can be adjusted based on the dataset (densy/sparse)):
# List of thresholds for each level (level 1 to max_level)
volume_threshold = []
occupancy_threshold = []
revenue_threshold = []

# Sales-oriented strategy with parameters:
# C1 Uniform strategy: alpha = 1/3, beta = 1/3, gamma = 1/3
# C2 Support-oriented strategy: alpha = 1.0, beta = 0.0, gamma = 0.
# C3 Occupancy-oriented strategy: alpha = 0.0, beta = 1.0, gamma =
# C4 Profit-oriented strategy: alpha = 0.0, beta = 0.0, gamma = 1.0
# C5 Important support: alpha = 0.5, beta = 0.25 gamma = 0.25
# C6 Important occupancy: alpha = 0.25, beta = 0.5, gamma = 0.25
# C7 Important profit: alpha = 0.25, beta = 0.25, gamma = 0.5
# C8 Important support and occupancy: alpha = 0.5, beta = 0.5, gamm
# C9 Important support and profit: alpha = 0.5, beta = 0.0, gamma =
# C10 Important occupancy and profit: alpha = 0.0, beta = 0.5, gamm
```

```
sales_strategies = {
    "C1": (1/3, 1/3, 1/3),
    "C2": (1.0, 0.0, 0.0),
    "C3": (0.0, 1.0, 0.0),
    "C4": (0.0, 0.0, 1.0),
    "C5": (0.5, 0.25, 0.25),
    "C6": (0.25, 0.5, 0.25),
    "C7": (0.25, 0.25, 0.5),
    "C8": (0.5, 0.5, 0.0),
    "C9": (0.5, 0.0, 0.5),
    "C10": (0.0, 0.5, 0.5)
}

index_case = 1 # Change value to select different strategy (C1 to C
current_Case = sales_strategies["C"+str(index_case)]
# Set of bias factor
alpha = current_Case[0]
beta = current_Case[1]
gamma = current_Case[2]

flags = {
    "C1": (1, 1, 1),      # tri-objectives
    "C2": (1, 0, 0),      # single-objective: support-oriented
    "C3": (0, 1, 0),      # single-objective: occupancy-oriented
    "C4": (0, 0, 1),      # single-objective: revenue-oriented
    "C5": (1, 1, 1),      # tri-objectives with support-oriented
    "C6": (1, 1, 1),      # tri-objectives with occupancy-oriented
    "C7": (1, 1, 1),      # tri-objectives with revenue-oriented
    "C8": (1, 1, 0),      # bi-objectives: support-oriented and occ
    "C9": (1, 0, 1),      # bi-objectives: support-oriented and rev
    "C10": (0, 1, 1)      # bi-objectives: occupancy-oriented and r
}
current_flags = flags["C"+str(index_case)]
# set of flags for objectives
flag_volume = current_flags[0]
flag_occupancy = current_flags[1]
flag_revenue = current_flags[2]

# Number of top itemsets to consider for TOI mining
k_factor = 5
```

```
The last running = 2026-10-14 17:50:08.055684
Time to load database from .dat: 8.215728044509888 seconds ---
Load Transaction database successfully
RAM usage after loading DB: 233.81 MB
```

Unique items

Denoted $\mathbf{I}$

$\mathbf{I} \leftarrow scan(\mathbf{D})$

Save I 'DBname/distinguished_item/distinguished_item_1.dat'

In [2]:
```
# Scan database for distinguished items
import time
import os
#STEP 2: LIST OF DISTINGUISHED ITEMS
import Func_unique
k=1
# Check if the file with distinguished items already exists
file_path = str(dbName)+"/distinguished_item/distinguished_item_"+s
if os.path.exists(file_path):
    # print("File exists")
    with open(file_path, 'r') as file:
        var_unique_items = list(file)
        var_unique_items = [int(line.strip()) for line in var_uniqu
else:
    # print("File does not exist")
    start_time = time.time()
    var_unique_items = Func_unique.Func_unique(DBtrans)
    print("Time to scan and obtain unique items: %s seconds ---" %
    f = open(str(dbName)+"/distinguished_item/distinguished_item_"+
    for item in var_unique_items:
      f.write(str(item) + "\n")
    f.close()
# print('List of distinguished items')
# print(var_unique_items)
print('Number of distinguished items: ', len(var_unique_items))
print(f"RAM usage after obtaining unique items: {memory_usage_mb():

var_unique_1_items = var_unique_items
```

```
Number of distinguished items:  32986
RAM usage after obtaining unique items: 234.62 MB
```

Log-normal profit

In [3]:
```
##
import numpy as np
import os
import matplotlib.pyplot as plt

file_path = str(dbName)+"/Profit.dat"
if os.path.exists(file_path):
    print("File exists")
    with open(file_path, 'r') as file:
        var_Profit = list(file)
        var_Profit = [float(line.strip()) for line in var_Profit]
else:
    np.random.seed(42)  # reproducible

    mu = 1.0      # mean of log
    sigma = 0.5   # std of log
    n = len(var_unique_items)      # number of samples
```

```
    Profit = np.random.lognormal(mean=mu, sigma=sigma, size=n)

    var_Profit = (Profit - Profit.min()) / (Profit.max() - Profit.m

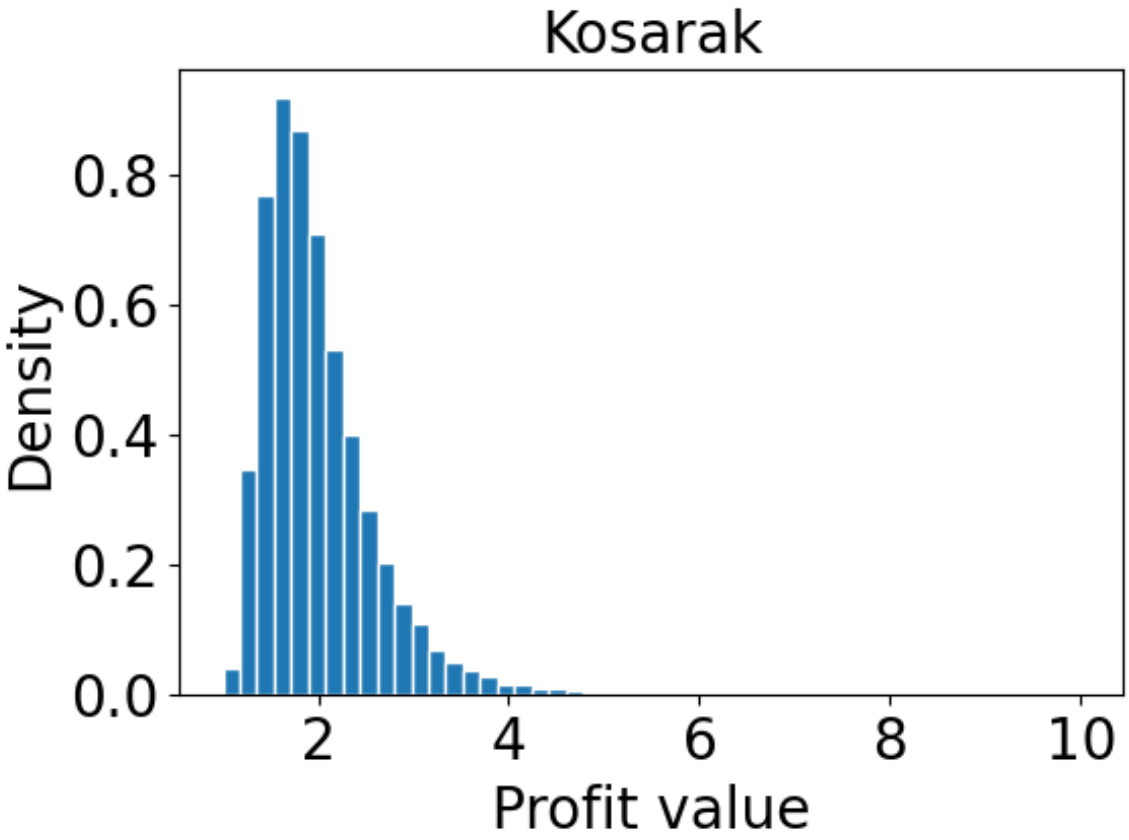
    var_Profit = 1 + 9 * var_Profit

    f = open(file_path, "w")
    for row in var_Profit:
      f.write(str(row)+str("\n"))
    f.close()

print(f"RAM usage after obtaining profit values: {memory_usage_mb()

plt.rcParams.update({
    # "font.family": "serif",
    # "font.serif": ["Times New Roman"],
    "font.size": 20,
    "axes.labelsize": 20,
    "axes.titlesize": 20,
    "xtick.labelsize": 20,
    "ytick.labelsize": 20,
    "legend.fontsize": 20
})
plt.figure(figsize=(6,4))
plt.tight_layout()
plt.hist(var_Profit,
         bins=50,
         density=True,
         edgecolor='white'
         )
plt.xlabel("Profit value")
plt.ylabel("Density")
plt.title(f"{dbName}")
plt.savefig(f"Lognormal_Profit_{dbName}.pdf", dpi=300, bbox_inches=
plt.show()
```

```
File exists
RAM usage after obtaining profit values: 266.89 MB
```



Random volume of item in a transaction

In [4]:
```
import numpy as np
import os

var_Volume = []
file_path = os.path.join(dbName, "Volume.dat")

os.makedirs(dbName, exist_ok=True)

if os.path.exists(file_path):
    print("File exists")

    with open(file_path, 'r', encoding='utf-8') as file:
        var_Volume = [
            list(map(int, line.split()))
            for line in file if line.strip()
        ]

else:
    for transaction in DBtrans:
        n = len(transaction)
        vol = np.random.poisson(lam=3, size=n)
        vol[vol == 0] = 1
        var_Volume.append(
            # np.random.randint(1, 10, size=n).tolist()
            vol.tolist()
        )
```

```
    with open(file_path, "w", encoding='utf-8') as f:
        for row in var_Volume:
            f.write(" ".join(map(str, row)) + "\n")

print(f"RAM usage to random volume values: {memory_usage_mb():.2f} |
```

```
File exists
RAM usage to random volume values: 343.12 MB
```

Statistic length of transactions

$|\mathbf{D}|$

$|\mathbf{T}_i|$

$\min(|\mathbf{T}_i|)$

$\max(|\mathbf{T}_i|)$

$Avg(|\mathbf{T}_i|)$

In [5]:
```
#STEP 3: LENGTH OF TRANSACTION
import statistics

import Func_length_of_transaction
file_path = str(dbName)+"/length_of_transaction_via_volume.dat"
file_path2 = str(dbName)+"/length_of_transaction.dat"
# check if the file with length of transactions already exists
if os.path.exists(file_path) and os.path.exists(file_path2):
    # print("File exists")
    with open(file_path, 'r') as file:
        var_volume_transactions = list(file)
        var_volume_transactions = [int(line.strip()) for line in va
    with open(file_path2, 'r') as file2:
        var_length_transactions = list(file2)
        var_length_transactions = [int(line.strip()) for line in va
else:
    start_time = time.time()
    var_volume_transactions = Func_length_of_transaction.Func_lengt
    var_length_transactions = Func_length_of_transaction.Func_lengt
    print("Time to scan and obtain length of transactions: %s secon
    # save file "length_of_transaction.dat"
    f = open(file_path, "w")
    for length in var_volume_transactions:
      f.write(str(length) + "\n")
    f.close()

    f2 = open(file_path2, "w")
    for length in var_length_transactions:
      f2.write(str(length) + "\n")
    f2.close()
```

```
print('Number of transactions: ', len(var_volume_transactions))
min_transaction_volume = min(var_volume_transactions)
max_transaction_volume = max(var_volume_transactions)
var_mean_transaction_volume = statistics.mean(var_volume_transactio
var_variance_transaction_volume = statistics.variance(var_volume_tr
var_sum_transaction_volume = sum(var_volume_transactions)

print("Max transaction volume: ", max_transaction_volume)
print("Min transaction volume: ", min_transaction_volume)
print("Avg transaction volume: ", var_mean_transaction_volume)
print("Variance transaction volume: ",var_variance_transaction_volu
print("Sum transaction volume: ", var_sum_transaction_volume)
print("Median transaction volume: ", statistics.median(var_volume_t
print("=" * 50)
print("Max transaction length: ", max(var_length_transactions))
print("Min transaction length: ", min(var_length_transactions))
var_mean_length_transaction = statistics.mean(var_length_transactio
print("Avg transaction length: ", var_mean_length_transaction)
print("Variance transaction length: ", statistics.variance(var_leng
print("Sum transaction length: ", sum(var_length_transactions))
median_length_transaction = statistics.median(var_length_transactio
print("Median transaction length: ", median_length_transaction)
max_level = round(np.log2(1+ var_mean_length_transaction * median_l
print("Max level for mining: ", max_level)
rho = (len(var_unique_items) / len(var_length_transactions) * max_l
       - (0 * (len(var_unique_items) / len(var_length_transactions)
)
print("Retention rho =", rho)
print(f"RAM usage after obtaining length of transactions: {memory_u
```

```
Number of transactions:  345468
Max transaction volume:  7540
Min transaction volume:  1
Avg transaction volume:  24.744653629279703
Variance transaction volume:  5248.780742806848
Sum transaction volume:  8548486
Median transaction volume:  10.0
==================================================
Max transaction length:  2498
Min transaction length:  1
Avg transaction length:  8.11468500700499
Variance transaction length:  560.8973481693547
Sum transaction length:  2803364
Median transaction length:  3.0
Max level for mining:  5
Retention rho = 0.4774103534914956
RAM usage after obtaining length of transactions: 356.91 MB
```

Dense/ Sparse dataset evaluation

$$\omega = \frac{\sum_{i=1}^{|\mathbf{D}|} |\mathbf{T}_i|}{|\mathbf{D}| \cdot |\mathbf{I}|}$$

In [6]:
```
# Calculate density/sparsity omega
omega = sum(var_length_transactions) / (len(var_length_transactions
print(f"Dense/sparse \u03C9 = {omega}")
```

```
Dense/sparse ω = 0.0002460039109623777
```

TDS generation and Robust threshold

$\eta_{\text{sup}}^{(1)}, \mu_{\text{occ}}^{(1)}, \varphi_{\text{pro}}^{(1)}$

In [7]:
```
import gc
import numpy as np
from collections import defaultdict
import json
from pathlib import Path
import time
import matplotlib.pyplot as plt

print(f"RAM usage before generating TDS: {memory_usage_mb():.2f} MB
# ----------------------------
# Step 1: Build TDS
# ----------------------------
profit_dict = {
    item_id: var_Profit[i]
    for i, item_id in enumerate(var_unique_items)
}

start_time = time.time()
TDS = defaultdict(lambda: [set(),[], set(), 0, 0.0, 0.0, 0.0, 0.0,
# [[sorted itemset],[uprofit], {(tid, volume item in tran, len tran

for tid, transaction in enumerate(DBtrans):
    total_volume_of_tran = var_volume_transactions[tid]  # Total vo

    if total_volume_of_tran == 0:
        continue

    for idx, item in enumerate(transaction):
        if item not in profit_dict:
            continue

        key = (item,)
        volume = var_Volume[tid][idx]
        occ = (volume/total_volume_of_tran
               + (
                   (total_volume_of_tran - min_transaction_volume)
                   )
               * (1 - np.exp(-(total_volume_of_tran - volume)/total
        )
        TDS[key][0] = key # sorted itemset
        TDS[key][1] = [profit_dict[item]] # unit profit
        TDS[key][2].add((tid, volume, total_volume_of_tran))
        TDS[key][3] += volume              # volume of item in tran
        TDS[key][4] += occ                # occupancy
```

```
        TDS[key][5] += volume * profit_dict[item] # Revenue of item
# Delete dataset to save memory, free up RAM
DBtrans.clear()
del DBtrans
gc.collect()

volume_vals    = np.array([x[3] for x in TDS.values()])  # volume
occupancy_vals = np.array([x[4] for x in TDS.values()])  # occupanc
revenue_vals   = np.array([x[5] for x in TDS.values()])  # revenue
# print(volume_vals)
# print(occupancy_vals)
# print(revenue_vals)

min_volume = np.min(volume_vals)
print("Min volume   : ", min_volume)
max_volume = np.max(volume_vals)
print("Max volume   : ", max_volume)
min_occupancy = np.min(occupancy_vals)
print("Min occupancy: ", min_occupancy)
max_occupancy = np.max(occupancy_vals)
print("Max occupancy: ", max_occupancy)
min_revenue = np.min(revenue_vals)
print("Min revenue  : ", min_revenue)
max_revenue = np.max(revenue_vals)
print("Max revenue  : ", max_revenue)

normalized_volume = (volume_vals - min_volume) / (max_volume - min_
normalized_occupancy = (occupancy_vals - min_occupancy) / (max_occu
normalized_revenue = (revenue_vals - min_revenue) / (max_revenue -

# items = list(TDS.keys())
# item_to_idx = {item: i for i, item in enumerate(items)}

for i, (item, vals) in enumerate(TDS.items()):
    TDS[item][6] = normalized_volume[i]  # norm volume
    TDS[item][7] = normalized_occupancy[i]
    TDS[item][8] = normalized_revenue[i]
    TDS[item][9] = (
        flag_volume * alpha * normalized_volume[i]
        + flag_occupancy * beta * normalized_occupancy[i]
        + flag_revenue * gamma * normalized_revenue[i]
        )

print("Time to build TDS: %s seconds ---" % (time.time() - start_ti

#################################################
# Drawing sales performance
items_drawing =             [str(item) for item, _ in TDS.items()]
support_vals_drawing =      [vals[3] for _, vals in TDS.items()]
occupancy_vals_drawing =    [vals[4] for _, vals in TDS.items()]
revenue_vals_drawing =      [vals[5] for _, vals in TDS.items()]

if dbName == 'Test':
    x = np.arange(len(items_drawing))
    width = 0.25
```

```
    plt.figure(figsize=(8, 6))

    plt.bar(x - width, support_vals_drawing, width,
            label='Sales-volume performance',
            edgecolor='black',
            color='#1f77b4',
            hatch='///')   # pattern 1

    plt.bar(x, occupancy_vals_drawing, width,
            label='Cross-selling performance',
            edgecolor='black',
            color ='green',
            hatch='xxx')   # pattern 2

    plt.bar(x + width, revenue_vals_drawing, width,
            label='Sales-revenue performance',
            edgecolor='black',
            color='coral',
            hatch='...')   # pattern 3

    # labels = [r'($a$)', r'($b$)', r'($c$)', r'($d$)', r'($e$)', r
    # plt.xticks(x, labels, rotation=0)
    plt.xticks(x, var_unique_items, rotation=0)
    # plt.xticks(x, items_drawing, rotation=45)
    plt.xlabel("Items")
    plt.ylabel("Sales performance")
    plt.title("Volume vs Occupancy vs Revenue")
    plt.legend()
    plt.tight_layout()
    plt.savefig(
        f"Sales_Performance_{dbName}.pdf",
        dpi=300,
        bbox_inches="tight"
    )
    plt.show()

##########################################
## Before sorting by TOI score
# The following code is commented out to save time and memory, sinc
# print("Before sorting by TOI score:")
# for item, vals in TDS.items():  # Print the first 5 items in TDS
#     print(f"""Item: {item}, Unit Profit: {vals[1]}, Support-list:
#           Volume: {vals[3]}, Occupancy: {vals[4]}, Revenue: {vals
#           Norm Volume: {vals[6]}, Norm Occupancy: {vals[7]}, Norm

## After sorting by TOI score
# print("After sorting by TOI score:")
# TDS = dict(sorted(TDS.items(), key=lambda x: x[1][9], reverse=Tru
# for item, vals in TDS.items():
#     print(f"""Item: {item}, Unit Profit: {vals[1]}, Support-list:
#           Support: {vals[3]}, Occupancy: {vals[4]}, Revenue: {val
#           Norm Volume: {vals[6]}, Norm Occupancy: {vals[7]}, Norm

# MAD (Median Absolute Deviation) function
def mad(arr):
    med = np.median(arr)
```

```
    return np.median(np.abs(arr - med))

# Calculate median and MAD for each measure
med_vol, mad_vol = np.median(volume_vals), mad(volume_vals)
med_occ, mad_occ = np.median(occupancy_vals), mad(occupancy_vals)
med_rev, mad_rev = np.median(revenue_vals), mad(revenue_vals)

lambdas = np.arange(-0.5, 1.6, 0.1)

best_lambda_support = None
best_diff = float("inf")

for lam in lambdas:
    theta = med_vol + lam * mad_vol
    ratio = np.sum(volume_vals >= theta) / len(volume_vals)
    diff = abs(ratio - rho)
    if diff < best_diff:
        best_diff = diff
        best_lambda_support = lam
#################
best_lambda_occupancy = None
best_diff = float("inf")

for lam in lambdas:
    theta = med_occ + lam * mad_occ
    ratio = np.sum(occupancy_vals >= theta) / len(occupancy_vals)
    diff = abs(ratio - rho)
    if diff < best_diff:
        best_diff = diff
        best_lambda_occupancy = lam

#################
best_lambda_profit = None
best_diff = float("inf")

for lam in lambdas:
    theta = med_rev + lam * mad_rev
    ratio = np.sum(revenue_vals >= theta) / len(revenue_vals)
    diff = abs(ratio - rho)
    if diff < best_diff:
        best_diff = diff
        best_lambda_profit = lam
# print(best_lambda_occupancy, med_occ + best_lambda_occupancy * ma

# Robust thresholds for support, occupancy, and profit
support_threshold_1   = med_vol + best_lambda_support * mad_vol
occupancy_threshold_1 = med_occ + best_lambda_occupancy * mad_occ
profit_threshold_1    = med_rev + best_lambda_profit * mad_rev

print("Robust thresholds    :")
print(f"Volume threshold    : {support_threshold_1}")
print(f"Occupancy threshold : {occupancy_threshold_1}")
print(f"Revenue threshold   : {profit_threshold_1}")


filtered_TDS = {
```

```
    item: vals
    for item, vals in TDS.items()
    if vals[3] >= flag_volume * support_threshold_1
    and vals[4] >= flag_occupancy * occupancy_threshold_1
    and vals[5] >= flag_revenue * profit_threshold_1
}

# After pre-pruning TDS based on robust thresholds
# print("After pre-pruning TDS based on robust thresholds:")
# for item, vals in filtered_TDS.items():  # Print the first 5 item
#     print(f"""Item: {item}, Unit Profit: {vals[1]}, Support-list:
#           Support: {vals[3]}, Occupancy: {vals[4]}, Revenue: {val
#           Norm Volume: {vals[6]}, Norm Occupancy: {vals[7]}, Norm

top_k_TOI = sorted(filtered_TDS.items(), key=lambda x: x[1][9], rev
print("Top-k TOIs:")
for item, vals in top_k_TOI:
    print(f"""Item: {item}, Unit Profit: {vals[1]}, No. Supports: {
          Volume: {vals[3]}, Occupancy: {vals[4]}, Revenue: {vals[5
          Norm Volume: {vals[6]}, Norm Occupancy: {vals[7]}, Norm R

total_running_time = time.time() - start_time
print("Time to obtain top-k TOIs: %s seconds ---" % (time.time() -

print(f"RAM after finishing level 1: {memory_usage_mb():.2f} MB")

TOI_nodes = [
    item for item, _ in top_k_TOI
]

set_TOI = set(top_k_TOI[i][0] for i in range(len(top_k_TOI)))
# print(f"Set of top {Top_item_set} items based on TOI mining: {set

avg_bias_support = np.mean([vals[6] for item, vals in top_k_TOI])
avg_bias_occupancy = np.mean([vals[7] for item, vals in top_k_TOI])
avg_bias_revenue = np.mean([vals[8] for item, vals in top_k_TOI])
print(f"Average bias for support among top {k_factor} items    : {a
print(f"Average bias for occupancy among top {k_factor} items  : {a
print(f"Average bias for revenue among top {k_factor} items    : {a
bias_matrix = []
bias_matrix.append([avg_bias_support, avg_bias_occupancy, avg_bias_

# From now on, we can delete the original TDS to save memory,
# since we will work with the filtered TDS for mining TOIs.
# del TDS
# gc.collect()
# print(f"RAM usage after deleting original TDS: {memory_usage_mb()
# Define thresholds for multiple levels
def Sequence_Thresholds(robust_threshold, omega, l_max):
    sequential_thresholds = [robust_threshold]
    factor = 1 / (1 + omega)

    for level in range(1, l_max):
        sequential_thresholds.append((sequential_thresholds[0] * fa

    return sequential_thresholds
```

```
volume_threshold = Sequence_Thresholds(support_threshold_1, omega, 
occupancy_threshold = Sequence_Thresholds(occupancy_threshold_1, om
revenue_threshold = Sequence_Thresholds(profit_threshold_1, omega, 

print("Sequential thresholds for support    :", [f"{x:.4f}" for x i
print("Sequential thresholds for occupancy  :", [f"{x:.4f}" for x i
print("Sequential thresholds for profit     :", [f"{x:.4f}" for x i
print(f"After pre-pruning, remaining itemsets: {len(filtered_TDS)}"


##################
## Save the filtered TDS to a JSONL file for later use in mining TO
file_path_2 = Path(f"TDS_{dbName}_case_{index_case}.jsonl")

with open(file_path_2, "w", encoding="utf-8") as f:
    for item, vals in filtered_TDS.items():

        record = {
            "item": list(item),
            "sort_itemset": list(item),
            "unitprofit": vals[1],
            "tidset": [
                {
                    "tid": int(tid),
                    "volume": int(vol),
                    "length": int(length)
                }
                for tid, vol, length in vals[2]
            ],

            "volume": int(vals[3]),
            "occupancy": float(vals[4]),
            "revenue": float(vals[5]),
            "norm_volume": float(vals[6]),
            "norm_occupancy": float(vals[7]),
            "norm_revenue": float(vals[8]),
            "combined_score": float(vals[9])
        }

        json.dump(record, f, ensure_ascii=False)
        f.write("\n")

# Draw distributions of support, occupancy, and profit with thresho
plt.rcParams.update({
    "font.size": 18,
    "axes.labelsize": 18,
    "xtick.labelsize": 18,
    "ytick.labelsize": 18
})
# ================================
# Create 1x3 subplots
fig, axes = plt.subplots(1, 3, figsize=(18, 3.5), sharey=True)

# Figure (a) Support
axes[0].hist(np.log10(volume_vals + 1),
```

```
             bins=40,
             density=True,
             edgecolor='white',
            color='#1f77b4',
            hatch='///')
axes[0].axvline(np.log10(support_threshold_1 + 1),
                linestyle='--', linewidth=2, color='red')
# axes[0].axvline(support_threshold_1,
#                 linestyle='--', linewidth=2, color='red')
axes[0].set_xlabel(r"$\log_{10}(Volume(\delta) + 1)$")
axes[0].set_ylabel("Probability Density")

axes[0].text(
    0.5, -0.29, "(a)",
    transform=axes[0].transAxes,
    ha="center", va="top", fontsize=18
)

# Figure (b) Occupancy
axes[1].hist(np.log10(occupancy_vals + 1),
             bins=40,
             density=True,
             edgecolor='white',
             color='green',
             hatch='xxx')
axes[1].axvline(np.log10(occupancy_threshold_1 + 1),
                linestyle='--', linewidth=2, color='red')
# axes[1].axvline(occupancy_threshold_1,
#                 linestyle='--', linewidth=2, color='red')
axes[1].set_xlabel(r"$\log_{10}(Occupancy(\delta) + 1)$")

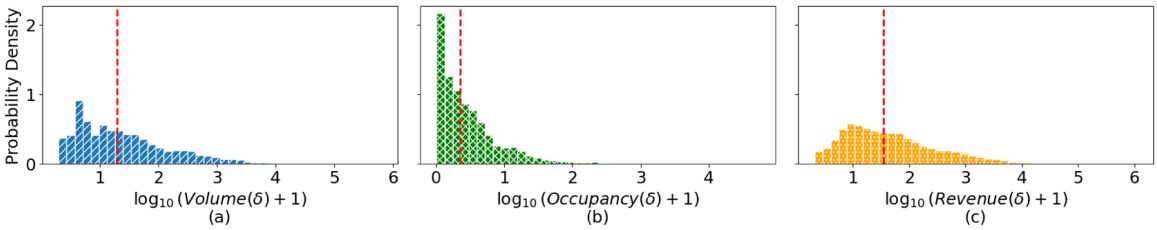
axes[1].text(
    0.5, -0.29, "(b)",
    transform=axes[1].transAxes,
    ha="center", va="top", fontsize=18
)

# Figure (c) Revenue
axes[2].hist(np.log10(revenue_vals + 1),
             bins=40,
             density=True,
             edgecolor='white',
             color='orange',
             hatch='...')
axes[2].axvline(np.log10(profit_threshold_1 + 1),
                linestyle='--', linewidth=2, color='red')
# axes[2].axvline(profit_threshold_1,
#                 linestyle='--', linewidth=2, color='red')
axes[2].set_xlabel(r"$\log_{10}(Revenue(\delta) + 1)$")

axes[2].text(
    0.5, -0.29, "(c)",
    transform=axes[2].transAxes,
    ha="center", va="top", fontsize=18
)
```

```
plt.tight_layout()
plt.subplots_adjust(bottom=0.25)  # Adjust bottom to fit x-labels
plt.savefig(
    f"Robust_Threshold_Distributions_{dbName}.pdf",
    dpi=300,
    bbox_inches="tight"
)
plt.show()
```

```
RAM usage before generating TDS: 359.06 MB
Min volume   :  1
Max volume   :  639286
Min occupancy:  0.018181188009574932
Max occupancy:  55176.92510395497
Min revenue  :  1.076039522847067
Max revenue  :  1124213.5157799537
Time to build TDS: 3.9087071418762207 seconds ---
Robust thresholds    :
Volume threshold    : 18.599999999999998
Occupancy threshold : 1.2501536489468867
Revenue threshold   : 33.80373574146906
Top-k TOIs:
Item: (6,), Unit Profit: [1.7585454957257347], No. Supports: 209858,
          Volume: 639286, Occupancy: 55176.92510395497, Revenue: 112
4213.5157799537,
          Norm Volume: 1.0, Norm Occupancy: 1.0, Norm Revenue: 1.0,
TOI Score: 1.0
Item: (3,), Unit Profit: [2.236488107936292], No. Supports: 157307,
          Volume: 478928, Occupancy: 50578.58393464071, Revenue: 107
1116.7765592528,
          Norm Volume: 0.7491603901233409, Norm Occupancy: 0.9166618
531961076, Norm Revenue: 0.9527698348250261, TOI Score: 0.8728640260
481582
Item: (11,), Unit Profit: [1.6651234314114147], No. Supports: 12685
9,
          Volume: 386284, Occupancy: 24206.938342150097, Revenue: 64
3210.5395792958,
          Norm Volume: 0.6042422393768038, Norm Occupancy: 0.4387146
998806105, Norm Revenue: 0.5721422756078766, TOI Score: 0.5383664049
550969
Item: (1,), Unit Profit: [2.1390254042175423], No. Supports: 68790,
          Volume: 209638, Occupancy: 17129.716749554555, Revenue: 44
8421.00768934283,
          Norm Volume: 0.32792416527839696, Norm Occupancy: 0.310450
5040912058, Norm Revenue: 0.39887472847512306, TOI Score: 0.34574979
928157523
Item: (4,), Unit Profit: [2.9725275953077115], No. Supports: 27061,
          Volume: 82840, Occupancy: 7881.621255013505, Revenue: 2462
44.18599526308,
          Norm Volume: 0.12958070344212674, Norm Occupancy: 0.142842
4229153992, Norm Revenue: 0.21903610140855162, TOI Score: 0.16381974
258869253
Time to obtain top-k TOIs: 3.926659107208252 seconds ---
RAM after finishing level 1: 708.06 MB
Average bias for support among top 5 items    : 0.5622
Average bias for occupancy among top 5 items  : 0.5617
Average bias for revenue among top 5 items    : 0.6286
Sequential thresholds for support    : ['18.6000', '9.2977', '6.198
5', '4.6489', '3.7191']
Sequential thresholds for occupancy  : ['1.2502', '0.6249', '0.416
6', '0.3125', '0.2500']
Sequential thresholds for profit     : ['33.8037', '16.8977', '11.26
51', '8.4489', '6.7591']
After pre-pruning, remaining itemsets: 13906
```



Baseline support calculation of single itemsets

$$\mathcal{S}up(\mathbf{S}) \leftarrow \mathcal{S}up(\mathbf{S}) \cup Tid, s.t. \quad \mathbf{T}_i \supseteq \mathbf{S}, i = 1, \ldots, |\mathbf{D}|.$$

and

$$|\mathcal{S}up\,(\mathbf{S})| = \sum_{i=1}^{|\mathbf{D}|} I, s.t. \quad \begin{bmatrix} I{=}1 | \mathbf{S} \subseteq \mathbf{T}_i \\ I{=}0 | \mathbf{S} \nsubseteq \mathbf{T}_i \end{bmatrix}.$$

Baseline Occupancy

$$Occupancy\,(\mathbf{S}) = \sum_{\mathbf{S} \in \mathbf{T}_i} \frac{|\mathbf{S}|}{|\mathbf{T}_i|}$$

Length-rewarded Occupancy

$$\mathcal{O}cc\,(\mathbf{S}) = \sum\nolimits_{\forall \mathbf{T}_i \in \mathcal{S}up(\mathbf{S})} \frac{|\mathbf{S}|}{|\mathbf{T}_i|} + \left(1 - \exp\left(-\frac{|\mathbf{T}_i| - |\mathbf{S}|}{|\mathbf{T}_i|}\right)\right)$$

profit calculation of a single item (level-wise $\ell = 1$)

$$n\mathcal{P}ro = |\mathcal{S}up(\mathbf{S})| \cdot u\mathcal{P}ro(\mathbf{S})$$

Level-wise TOI mining

$$TOI(\mathbf{S}) = \alpha \cdot \mathcal{S}up(\mathbf{S}) + \beta \cdot \mathcal{O}cc(\mathbf{S}) + \gamma \cdot n\mathcal{P}ro(\mathbf{S})$$

where:

1. $\mathcal{S}up(\mathbf{S}) \geq \eta_{\text{sup}}$.
2. $\mathcal{O}cc(\mathbf{S}) \geq \mu_{\text{occ}}$.
3. $n\mathcal{P}ro(\mathbf{S}) \geq \varphi_{\text{pro}}$

In [8]:
```python
import numpy as np
import time
from collections import defaultdict
import heapq
import json
import os

# ===================== Encode TIDs =====================
all_tids = set()
for vals in TDS.values(): # filtered_TDS
    for tid, _, _ in vals[2]:
        all_tids.add(tid)

tid_to_idx = {tid: i for i, tid in enumerate(sorted(all_tids))}
idx_to_tid = {i: tid for tid, i in tid_to_idx.items()}
N = len(tid_to_idx)

# ===================== STEP 2: Build bitset + data ===============
bitset_dict = {}
data_dict = {}

for item, vals in TDS.items(): # filtered_TDS
    bitset = np.zeros(N, dtype=np.bool_)
    data = {}

    for tid, vol, l in vals[2]:
        idx = tid_to_idx[tid]
        bitset[idx] = True
        data[idx] = (vol if isinstance(vol, list) else [vol], l)

    bitset_dict[item] = bitset
    data_dict[item] = data

# ===================== INIT =====================
current_level = 2
next_top_k_TOI = dict(top_k_TOI)

bitset_cache = {}
data_cache = {}

# init cache từ level 1
for item, vals in next_top_k_TOI.items():
    base_item = item if isinstance(item, tuple) else (item,)
    b = np.zeros(N, dtype=np.bool_)
    d = {}

    for tid, vol, l in vals[2]:
        idx = tid_to_idx[tid]
        b[idx] = True
        d[idx] = (vol if isinstance(vol, list) else [vol], l)
```

```
    bitset_cache[base_item] = b
    data_cache[base_item] = d

# ===================== MAIN LOOP =====================
while next_top_k_TOI and current_level <= max_level:
    start_time = time.time()
    seen_itemsets = set()

    current_top_k_TOI = defaultdict(
        lambda: [set(), [], None, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0
    )

    for itemset, values in next_top_k_TOI.items():
        base_items = itemset if isinstance(itemset, tuple) else (it

        bitA = bitset_cache[base_items]
        dataA = data_cache[base_items]

        for item, value in TDS.items(): # filtered_TDS
            if item[0] in base_items:
                continue

            new_items = base_items + item
            key = frozenset(new_items)

            if key in seen_itemsets:
                continue
            seen_itemsets.add(key)

            bitB = bitset_dict[item]
            dataB = data_dict[item]

            # BITSET INTERSECTION
            common_bits = bitA & bitB

            if not common_bits.any():
                continue

            common_idx = np.nonzero(common_bits)[0]

            list_of_supports = []
            append = list_of_supports.append

            for idx in common_idx:
                volA, lenTid = dataA[idx]
                volB, _ = dataB[idx]

                new_vol = volB + volA
                append((idx, new_vol, lenTid))

            if not list_of_supports:
                continue

            profit_itemset = values[1] + value[1]

            # ===================== METRICS =====================
```

```
            volume = 0.0
            occupancy = 0.0
            revenue = 0.0

            for _, vol_itemset, lenTid in list_of_supports:
                s = sum(vol_itemset)
                volume += s
                occupancy += (s / lenTid
                              + (
                                  (lenTid - min_transaction_volume)
                                  )
                               * (1 - np.exp(-(lenTid - s) / lenTid
                )

                for j in range(len(vol_itemset)):
                    revenue += profit_itemset[j] * vol_itemset[j]

            if (
                volume < flag_volume * volume_threshold[current_lev
                or occupancy < flag_occupancy * occupancy_threshold
                or revenue < flag_revenue * revenue_threshold[curre
            ):
                continue

            # CACHE BITSET + DATA
            new_data = {
                idx: (vol, lenTid)
                for idx, vol, lenTid in list_of_supports
            }

            bitset_cache[new_items] = common_bits
            data_cache[new_items] = new_data

            current_top_k_TOI[new_items][0] = new_items
            current_top_k_TOI[new_items][1] = profit_itemset
            current_top_k_TOI[new_items][2] = list_of_supports
            current_top_k_TOI[new_items][3] = volume
            current_top_k_TOI[new_items][4] = occupancy
            current_top_k_TOI[new_items][5] = revenue

    if not current_top_k_TOI:
        break

    # ===================== NORMALIZE =====================
    volumes = [vals[3] for vals in current_top_k_TOI.values()]
    occupancies = [vals[4] for vals in current_top_k_TOI.values()]
    revenues = [vals[5] for vals in current_top_k_TOI.values()]

    min_volume, max_volume = min(volumes), max(volumes)
    min_occupancy, max_occupancy = min(occupancies), max(occupancie
    min_revenue, max_revenue = min(revenues), max(revenues)

    for item, vals in current_top_k_TOI.items():
        vals[6] = (vals[3] - min_volume) / (max_volume - min_volume
        vals[7] = (vals[4] - min_occupancy) / (max_occupancy - min_
        vals[8] = (vals[5] - min_revenue) / (max_revenue - min_reve
```

```
        vals[9] = (flag_volume * alpha * vals[6]
                   + flag_occupancy * beta * vals[7]
                   + flag_revenue * gamma * vals[8])

    # TOP-K
    current_top_k_TOI = dict(heapq.nlargest(
        k_factor,
        current_top_k_TOI.items(),
        key=lambda x: x[1][9]
    ))

    # ==================== OUTPUT ====================
    print("-" * 50)
    for item, vals in current_top_k_TOI.items():
        print(f"""Item: {item}, Unit Profit: {vals[1]}, No. Support
Volume: {vals[3]}, Occupancy: {vals[4]}, Revenue: {vals[5]},
Norm Volume: {vals[6]}, Norm Occupancy: {vals[7]}, Norm Revenue: {v

    TOI_nodes.extend([tuple(item) for item in current_top_k_TOI.key

    avg_bias_support = np.mean([vals[6] for vals in current_top_k_T
    avg_bias_occupancy = np.mean([vals[7] for vals in current_top_k
    avg_bias_revenue = np.mean([vals[8] for vals in current_top_k_T

    print(f"Average bias support   : {avg_bias_support:.4f}")
    print(f"Average bias occupancy : {avg_bias_occupancy:.4f}")
    print(f"Average bias revenue   : {avg_bias_revenue:.4f}")

    bias_matrix.append([avg_bias_support, avg_bias_occupancy, avg_b

    print("Running time:", time.time() - start_time)
    total_running_time += time.time() - start_time
    print(f"RAM level {current_level}: {memory_usage_mb():.2f} MB")

    next_top_k_TOI = current_top_k_TOI

    for r in next_top_k_TOI:
        set_TOI.add(r)

    current_level += 1

# ==================== FINAL ====================
print("=" * 50)
avg_bias_weights = np.mean(bias_matrix, axis=0)
print(f"Support  : {avg_bias_weights[0]:.4f}")
print(f"Occupancy: {avg_bias_weights[1]:.4f}")
print(f"Revenue  : {avg_bias_weights[2]:.4f}")
print("=" * 50)
print(f"Total running time for mining TOIs: {total_running_time:.2f
print(f"Final TOI: {set_TOI}")
print("=" * 50)
# ==================== SAVE ====================
file_path = f"Average_Bias_Weights_{dbName}.json"
key = "C" + str(index_case)
value = avg_bias_weights.tolist()
```

```
if os.path.exists(file_path):
    with open(file_path, "r") as f:
        data = json.load(f)
else:
    data = {}

data[key] = value

with open(file_path, "w") as f:
    json.dump(data, f, indent=4)
```

```
--------------------------------------------------
Item: (6, 11), Unit Profit: [1.7585454957257347, 1.665123431411414
7], No. Supports: 112773,
Volume: 687359.0, Occupancy: 35940.99525638265, Revenue: 1176613.575
3628586,
Norm Volume: 1.0, Norm Occupancy: 0.9519028935754711, Norm Revenue:
1.0, TOI Score: 0.9839676311918235
Item: (6, 3), Unit Profit: [1.7585454957257347, 2.236488107936292],
No. Supports: 92583,
Volume: 564170.0, Occupancy: 37756.9660103769, Revenue: 1126940.9658
598474,
Norm Volume: 0.8207766360320594, Norm Occupancy: 1.0, Norm Revenue:
0.9577828061997599, TOI Score: 0.9261864807439397
Item: (3, 11), Unit Profit: [2.236488107936292, 1.6651234314114147],
No. Supports: 56275,
Volume: 343098.0, Occupancy: 17831.423813255216, Revenue: 669440.972
734169,
Norm Volume: 0.4991467216799617, Norm Occupancy: 0.4722597148035126
4, Norm Revenue: 0.5689494820269189, TOI Score: 0.5134519728367977
Item: (6, 1), Unit Profit: [1.7585454957257347, 2.1390254042175423],
No. Supports: 46033,
Volume: 280000.0, Occupancy: 13754.155057025344, Revenue: 545580.786
1710362,
Norm Volume: 0.407347650174802, Norm Occupancy: 0.36427073488274003,
Norm Revenue: 0.46367960487354143, TOI Score: 0.41176599664369445
Item: (3, 1), Unit Profit: [2.236488107936292, 2.1390254042175423],
No. Supports: 29412,
Volume: 178900.0, Occupancy: 11086.582993134818, Revenue: 391411.320
38254844,
Norm Volume: 0.2602607990991476, Norm Occupancy: 0.2936184420691909
7, Norm Revenue: 0.3326496010063695, TOI Score: 0.295509614058236
Average bias support   : 0.5975
Average bias occupancy : 0.6164
Average bias revenue   : 0.6646
Running time: 24.369187831878662
RAM level 2: 25749.92 MB
--------------------------------------------------
Item: (6, 11, 3), Unit Profit: [1.7585454957257347, 1.66512343141141
47, 2.236488107936292], No. Supports: 49991,
Volume: 457219.0, Occupancy: 20698.33012844752, Revenue: 862686.2900
848192,
Norm Volume: 1.0, Norm Occupancy: 1.0, Norm Revenue: 1.0, TOI Score:
1.0
Item: (6, 11, 1), Unit Profit: [1.7585454957257347, 1.66512343141141
47, 2.1390254042175423], No. Supports: 29913,
```

Volume: 273564.0, Occupancy: 8929.553402590274, Revenue: 507222.4057
1933804,
Norm Volume: 0.5983154422893537, Norm Occupancy: 0.4314027453444925,
Norm Revenue: 0.5879515647213238, TOI Score: 0.5392232507850566
Item: (6, 3, 1), Unit Profit: [1.7585454957257347, 2.23648810793629
2, 2.1390254042175423], No. Supports: 20129,
Volume: 183700.0, Occupancy: 7794.32264886831, Revenue: 375504.80526
09605,
Norm Volume: 0.40176767013989134, Norm Occupancy: 0.3765551511549788
4, Norm Revenue: 0.4352664859982972, TOI Score: 0.4045297690977224
Item: (6, 11, 7), Unit Profit: [1.7585454957257347, 1.66512343141141
47, 3.031686583839584], No. Supports: 19394,
Volume: 177689.0, Occupancy: 4352.638857952816, Revenue: 382237.8918
2381204,
Norm Volume: 0.38862059613483463, Norm Occupancy: 0.2102734720568170
7, Norm Revenue: 0.4430713795608914, TOI Score: 0.3473218159175143
Item: (6, 11, 218), Unit Profit: [1.7585454957257347, 1.665123431411
4147, 1.7573897857150649], No. Supports: 21162,
Volume: 193454.0, Occupancy: 4288.357490910122, Revenue: 334097.2565
471738,
Norm Volume: 0.4231013184255881, Norm Occupancy: 0.2071677789438128
9, Norm Revenue: 0.38726747285889584, TOI Score: 0.3391788567427656
Average bias support : 0.5624
Average bias occupancy : 0.4451
Average bias revenue : 0.5707
Running time: 19.422698974609375
RAM level 3: 49125.73 MB

Item: (6, 11, 218, 148), Unit Profit: [1.7585454957257347, 1.6651234
314114147, 1.7573897857150649, 1.3970728264833518], No. Supports: 17
412,
Volume: 212276.0, Occupancy: 3891.3253939770416, Revenue: 349104.136
03791426,
Norm Volume: 1.0, Norm Occupancy: 0.8169879330529137, Norm Revenue:
1.0, TOI Score: 0.9389959776843044
Item: (6, 11, 3, 1), Unit Profit: [1.7585454957257347, 1.66512343141
14147, 2.236488107936292, 2.1390254042175423], No. Supports: 13085,
Volume: 159590.0, Occupancy: 4762.9445227754195, Revenue: 311317.321
1024867,
Norm Volume: 0.7517984086380146, Norm Occupancy: 1.0, Norm Revenue:
0.8917579507689632, TOI Score: 0.8811854531356593
Item: (6, 11, 7, 27), Unit Profit: [1.7585454957257347, 1.6651234314
114147, 3.031686583839584, 1.4413776211081983], No. Supports: 10546,
Volume: 129022.0, Occupancy: 2160.395410027205, Revenue: 254542.9731
5746223,
Norm Volume: 0.607793810741929, Norm Occupancy: 0.4535481419367368,
Norm Revenue: 0.7291252500285874, TOI Score: 0.5968224009024177
Item: (6, 11, 3, 7), Unit Profit: [1.7585454957257347, 1.66512343141
14147, 2.236488107936292, 3.031686583839584], No. Supports: 8601,
Volume: 104833.0, Occupancy: 2404.486433598674, Revenue: 227723.5279
2030768,
Norm Volume: 0.49384042097130554, Norm Occupancy: 0.504799429650705
7, Norm Revenue: 0.652299729892265, TOI Score: 0.5503131935047587
Item: (6, 11, 3, 218), Unit Profit: [1.7585454957257347, 1.665123431
4114147, 2.236488107936292, 1.7573897857150649], No. Supports: 9297,
Volume: 113264.0, Occupancy: 2399.444695656298, Revenue: 210085.7852

```
2492692,
Norm Volume: 0.5335585171785123, Norm Occupancy: 0.5037408263555808,
Norm Revenue: 0.6017756141016056, TOI Score: 0.5463583192118995
Average bias support   : 0.6774
Average bias occupancy : 0.6558
Average bias revenue   : 0.7750
Running time: 15.91533374786377
RAM level 4: 50260.25 MB
--------------------------------------------------
Item: (6, 11, 218, 148, 3), Unit Profit: [1.7585454957257347, 1.6651
234314114147, 1.7573897857150649, 1.3970728264833518, 2.236488107936
292], No. Supports: 7511,
Volume: 114327.0, Occupancy: 2000.4636193596043, Revenue: 201476.620
4550878,
Norm Volume: 0.9490216913077044, Norm Occupancy: 1.0, Norm Revenue:
0.9589572451011352, TOI Score: 0.9693263121362798
Item: (6, 11, 218, 148, 1), Unit Profit: [1.7585454957257347, 1.6651
234314114147, 1.7573897857150649, 1.3970728264833518, 2.139025404217
5423], No. Supports: 7884,
Volume: 120468.0, Occupancy: 1556.8408932154125, Revenue: 210099.310
39410093,
Norm Volume: 1.0, Norm Occupancy: 0.7782123293465908, Norm Revenue:
1.0, TOI Score: 0.9260707764488636
Item: (6, 11, 7, 27, 3), Unit Profit: [1.7585454957257347, 1.6651234
314114147, 3.031686583839584, 1.4413776211081983, 2.23648810793629
2], No. Supports: 4694,
Volume: 71763.0, Occupancy: 1137.3506858496205, Revenue: 145537.7024
5052603,
Norm Volume: 0.5956849821106895, Norm Occupancy: 0.5684896292445805,
Norm Revenue: 0.6926960995412973, TOI Score: 0.6189569036321891
Item: (6, 11, 218, 148, 7), Unit Profit: [1.7585454957257347, 1.6651
234314114147, 1.7573897857150649, 1.3970728264833518, 3.031686583839
584], No. Supports: 4216,
Volume: 64298.0, Occupancy: 752.1902920577493, Revenue: 123750.58937
37598,
Norm Volume: 0.5337157467438135, Norm Occupancy: 0.3759300024880301,
Norm Revenue: 0.5889925977004423, TOI Score: 0.4995461156440953
Item: (6, 11, 3, 1, 218), Unit Profit: [1.7585454957257347, 1.665123
4314114147, 2.236488107936292, 2.1390254042175423, 1.757389785715064
9], No. Supports: 3803,
Volume: 57954.0, Occupancy: 828.6700412022354, Revenue: 110798.17833
779517,
Norm Volume: 0.48105227331213735, Norm Occupancy: 0.414165792530489
2, Norm Revenue: 0.5273409976477458, TOI Score: 0.47418635449679075
Average bias support   : 0.7119
Average bias occupancy : 0.6274
Average bias revenue   : 0.7536
Running time: 13.195817947387695
RAM level 5: 50535.56 MB
==================================================
Support  : 0.6223
Occupancy: 0.5813
Revenue  : 0.6785
==================================================
Total running time for mining TOIs: 76.83 seconds
Final TOI: {(6, 11, 3, 1), (6, 11, 3, 7), (11,), (3, 1), (6, 11, 21
```

```
8, 148), (6, 11, 3, 1, 218), (6, 11, 3), (6, 11), (4,), (1,), (6, 3,
1), (6, 11, 218, 148, 1), (6, 11, 218, 148, 7), (6, 11, 7, 27, 3), (
6, 1), (6, 11, 7, 27), (3,), (6, 11, 218), (3, 11), (6,), (6, 11, 21
8, 148, 3), (6, 11, 3, 218), (6, 11, 1), (6, 11, 7), (6, 3)}
================================================
```

Sales performance evaluatiion

In [9]:
```python
import json
import os

print("Sales performance:")
unique_TOI = {item for t in set_TOI for item in t}
print(f"Unique items in TOI: {unique_TOI}")
print(f"Item rate = {len(unique_TOI)}/{len(var_unique_items)} = {le
total_TOI_volume = 0
total_TOI_revenue = 0.0
for item in unique_TOI:
    if (item,) in TDS:
        vals = TDS[(item,)]
        total_TOI_volume += vals[3]
        total_TOI_revenue += vals[5]
total_volume = sum(vals[3] for vals in TDS.values())
total_revenue = sum(vals[5] for vals in TDS.values())
print(f"Total sales volume of TOI : {total_TOI_volume}")
print(f"Sales volume rate of TOI  : {total_TOI_volume/total_volume:
print(f"Total sales revenue of TOI: {total_TOI_revenue:.4f}")
print(f"Sales revenue rate of TOI : {total_TOI_revenue/total_revenu

file_path = f"Sales_Performance_{dbName}_{index_case}_Retention_{rh
key = "C" + str(index_case)
value = [
    list(unique_TOI),                       # unique items in TOI
    len(unique_TOI)/len(var_unique_items),  # item rate
    total_TOI_volume,                       # total volume of TOI
    total_TOI_revenue,                     # total revenue of TOI
    total_TOI_volume/total_volume,          # volume rate
    total_TOI_revenue/total_revenue         # revenue rate
]

if os.path.exists(file_path):
    with open(file_path, "r") as f:
        data = json.load(f)
else:
    data = {}

data[key] = value
with open(file_path, "w") as f:
    json.dump(data, f, indent=4)
#####################################################################
```

```
Sales performance:
Unique items in TOI: {1, 3, 4, 6, 7, 11, 148, 218, 27}
Item rate = 9/32986 = 0.0003
Total sales volume of TOI : 2134485
Sales volume rate of TOI  : 0.2497
Total sales revenue of TOI: 4193291.0549
Sales revenue rate of TOI : 0.2468
```

Radar chart of sales-oriented strategies

In [10]:
```python
import numpy as np
import matplotlib.pyplot as plt
import os
import json

# ============================================================
# LOAD DATA
# ============================================================

if os.path.exists(f"Average_Bias_Weights_{dbName}.json"):

    with open(f"Average_Bias_Weights_{dbName}.json", "r") as f:
        data = json.load(f)

else:

    data = {}

# ============================================================
# LABELS
# ============================================================

labels = np.array([
    'Volume',
    'Occupancy',
    'Revenue'
])

# ============================================================
# STATS
# ============================================================

stats = {
    r'$\mathcal{C}_1$': data["C1"] if "C1" in data else [0, 0, 0],
    r'$\mathcal{C}_2$': data["C2"] if "C2" in data else [0, 0, 0],
    r'$\mathcal{C}_3$': data["C3"] if "C3" in data else [0, 0, 0],
    r'$\mathcal{C}_4$': data["C4"] if "C4" in data else [0, 0, 0],
    r'$\mathcal{C}_5$': data["C5"] if "C5" in data else [0, 0, 0],
    r'$\mathcal{C}_6$': data["C6"] if "C6" in data else [0, 0, 0],
    r'$\mathcal{C}_7$': data["C7"] if "C7" in data else [0, 0, 0],
    r'$\mathcal{C}_8$': data["C8"] if "C8" in data else [0, 0, 0],
    r'$\mathcal{C}_9$': data["C9"] if "C9" in data else [0, 0, 0],
```

```
    r'$\mathcal{C}_{10}$': data["C10"] if "C10" in data else [0, 0,
}

# ============================================================
# ANGLES
# ============================================================

num_vars = len(labels)

angles = np.linspace(
    0,
    2 * np.pi,
    num_vars,
    endpoint=False
).tolist()

angles += angles[:1]

# ============================================================
# FIGURE
# ============================================================

fig, ax = plt.subplots(
    figsize=(8, 8),
    subplot_kw=dict(polar=True)
)

# ============================================================
# STYLE
# ============================================================

colors = [
    '#1f77b4',
    '#ff7f0e',
    '#2ca02c',
    '#d62728',
    '#9467bd',
    '#8c564b',
    '#e377c2',
    '#7f7f7f',
    '#bcbd22',
    '#17becf'
]

markers = [
    'D',
    'x',
    '+',
    'o',
    's',
    'P',
    '*',
    'H',
    'v',
    '^'
]
```

```
lines = [
    'solid',
    'dashed',
    'dotted',
    'dashdot',
    (0, (3, 1, 1, 1)),
    (0, (5, 1)),
    (0, (1, 1)),
    (0, (3, 5, 1, 5)),
    (0, (5, 10)),
    (0, (1, 10))
]

# ============================================================
# PLOT
# ============================================================

for i, (name, avg_bias_weights) in enumerate(stats.items()):

    values = avg_bias_weights + avg_bias_weights[:1]

    ax.plot(
        angles,
        values,
        color=colors[i],
        linestyle=lines[i],
        linewidth=1.5,
        marker=markers[i],
        label=name
    )

    ax.fill(
        angles,
        values,
        color=colors[i],
        alpha=0.1
    )

# ============================================================
# RADAR ORIENTATION
# ============================================================

ax.set_theta_offset(np.pi / 2)
ax.set_theta_direction(-1)

# ============================================================
# AXIS LABELS
# ============================================================

ax.set_thetagrids(
    np.degrees(angles[:-1]),
    labels
)

# ============================================================
```

```
# TĂNG KÍCH THƯỚC LABEL:
# Volume / Occupancy / Revenue
# ============================================================

for label in ax.get_xticklabels():

    label.set_fontsize(20)
    label.set_fontweight('bold')

# ============================================================
# Y AXIS RANGE
# ============================================================

ax.set_ylim(
    0.5,
    0.70
)

# ============================================================
# TĂNG KÍCH THƯỚC CÁC SỐ LIỆU
# TRÊN CÁC VÒNG RADAR
# ============================================================

ax.tick_params(
    axis='y',
    labelsize=20
)

# ============================================================
# LEGEND
# ============================================================

legend_handles, legend_labels = ax.get_legend_handles_labels()

legend = fig.legend(
    legend_handles,
    legend_labels,

    loc="lower center",

    title=f"{dbName}",
    title_fontsize=22,

    ncol=5,

    columnspacing=0.5,   # <-- giảm khoảng cách giữa các cột
    handletextpad=0.4,   # khoảng cách giữa ký hiệu và text

    frameon=True,
    facecolor="white",
    edgecolor="black",
    framealpha=1.0,

    bbox_to_anchor=(0.5, 0.08),

    fontsize=24
```

```
)
legend.get_title().set_fontweight("bold")


# ============================================================
# LAYOUT
# ============================================================

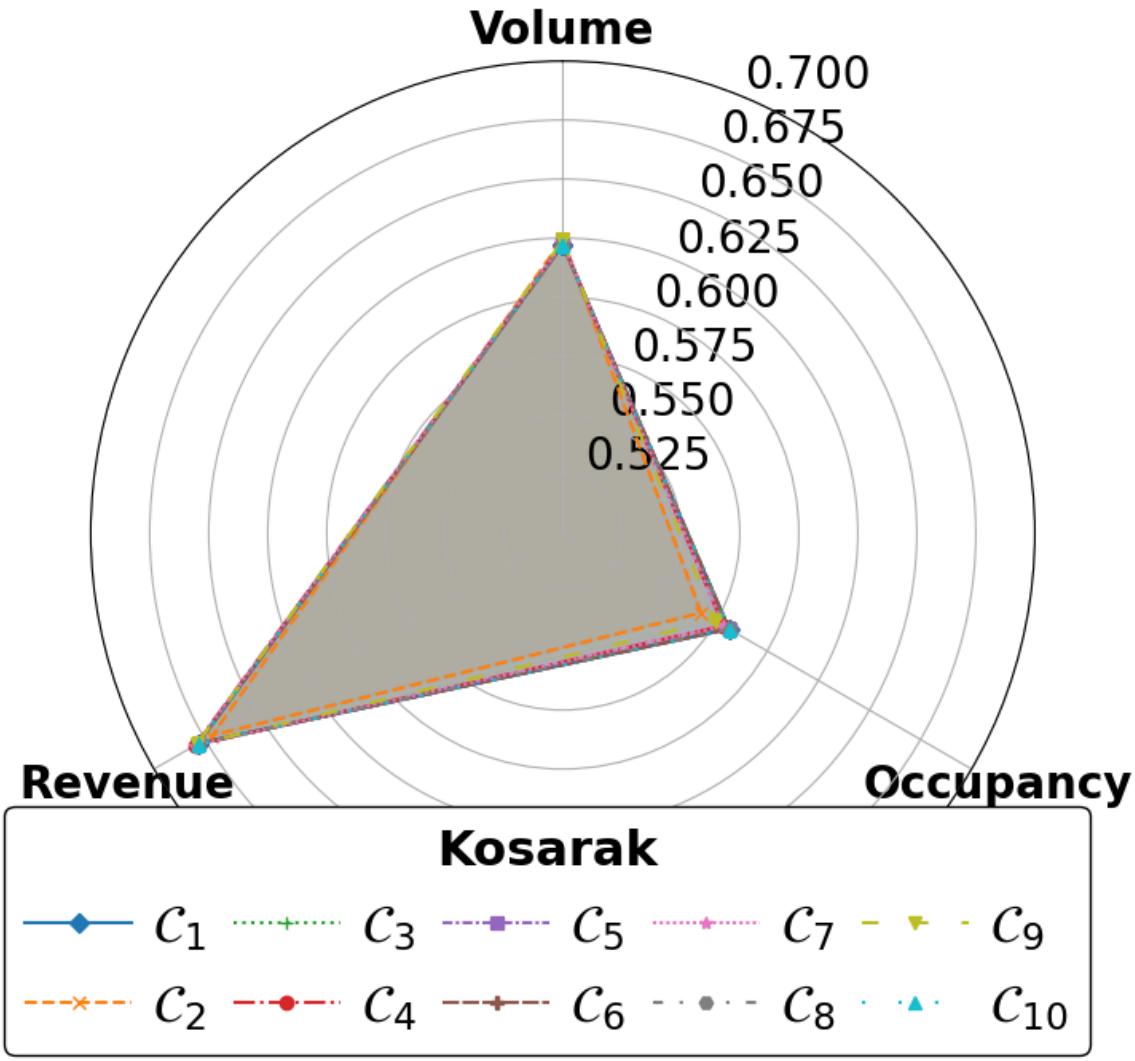
plt.subplots_adjust(
    bottom=0.14
)

# ============================================================
# SAVE
# ============================================================

plt.savefig(
    f"Radar_Chart_{dbName}.pdf",
    dpi=300,
    bbox_inches="tight"
)

# ============================================================
# SHOW
# ============================================================

plt.show()
```



Entropy-based weights

In [11]:
```python
import numpy as np


def entropy_based_thresholds(X):
    """
    Compute entropy-based thresholds for multi-criteria evaluation.

    Parameters
    ----------
    X : array-like of shape (m, k)
        Normalized decision matrix.
        m: number of bias strategies
        k: number of criteria (e.g., Support, Occupancy, Profit)

    Returns
    -------
    weights : numpy.ndarray of shape (k,)
        Entropy-based weights (thresholds) for each criterion.
    """
    X = np.array(X, dtype=float)
    m, k = X.shape
```

```
    # Step 1: probability distribution
    column_sums = X.sum(axis=0)
    P = X / column_sums

    # Step 2: entropy computation
    omega = 1.0 / np.log(m)
    entropy = -omega * np.sum(P * np.log(P), axis=0)

    # Step 3: diversification degree
    d = 1.0 - entropy

    # Step 4: entropy weights (thresholds)
    weights = d / np.sum(d)

    return weights


X = np.array(list(data.values()))
alpha, beta, gamma = entropy_based_thresholds(X)

print("Entropy-based weights (alpha, beta, gamma):")
print("alpha :", alpha)
print("beta  :", beta)
print("gamma :", gamma)
```

```
Entropy-based weights (alpha, beta, gamma):
alpha : 0.02406757792993288
beta  : 0.8160778545825013
gamma : 0.15985456748756588
```

The optimal sales-oriented strategy selection

In [12]:

```
import numpy as np

# 1. Data matrix (7 strategies x 3 criteria)
data = np.array(list(data.values()))

# POSITIVE CRITERIA (Benefit criteria)
is_benefit = [True, True, True]

def calculate_entropy_weights(matrix):
    # Normalize the decision matrix to get probabilities
    norm_matrix = matrix / matrix.sum(axis=0)

    # Calculate Entropy (e)
    k = 1.0 / np.log(len(matrix))
    # Avoid log(0) by adding a small constant (1e-12)
    entropy = -k * np.sum(norm_matrix * np.log(norm_matrix + 1e-12)
    # Calculate the degree of diversification (d)
    dispersion = 1 - entropy
    # Calculate weights (w)
    weights = dispersion / np.sum(dispersion)
```

```
    return weights

def calculate_topsis(matrix, weights, is_benefit):
    # Vector Normalization
    norm_matrix = matrix / np.sqrt(np.sum(matrix**2, axis=0))

    # Multiply by weights to get the weighted normalized decision m
    weighted_matrix = norm_matrix * weights

    # Define ideal best (PIS) and ideal worst (NIS)
    pis = []
    nis = []
    for j in range(len(is_benefit)):
        if is_benefit[j]:
            pis.append(np.max(weighted_matrix[:, j]))
            nis.append(np.min(weighted_matrix[:, j]))
        else:
            pis.append(np.min(weighted_matrix[:, j]))
            nis.append(np.max(weighted_matrix[:, j]))

    # Calculate distances to PIS and NIS (Euclidean distance)
    dp = np.sqrt(np.sum((weighted_matrix - pis)**2, axis=1))
    dn = np.sqrt(np.sum((weighted_matrix - nis)**2, axis=1))

    # Calculate Closeness Coefficient
    score = dn / (dp + dn)
    return score


weights = calculate_entropy_weights(data)
scores = calculate_topsis(data, weights, is_benefit)

# Display results
print("--- TOPSIS score ---")
print(f"Entropy-based weights: {weights}")
print("-" * 30)
for i, s in enumerate(scores):
    print(f"Sales-oriented strategy C{i+1}: TOPSIS = {s:.8f}")

# Find the best configuration
best_idx = np.argmax(scores)
print("-" * 30)
print(f"The optimal sales-oriented strategy: C{best_idx + 1}")
```

```
--- TOPSIS score ---
Entropy-based weights: [0.02406786 0.81607742 0.15985472]
------------------------------
Sales-oriented strategy C1: TOPSIS = 0.98463142
Sales-oriented strategy C2: TOPSIS = 0.00475324
Sales-oriented strategy C3: TOPSIS = 0.98391925
Sales-oriented strategy C4: TOPSIS = 0.80018586
Sales-oriented strategy C5: TOPSIS = 0.98463142
Sales-oriented strategy C6: TOPSIS = 0.98463142
Sales-oriented strategy C7: TOPSIS = 0.79910234
Sales-oriented strategy C8: TOPSIS = 0.98463569
Sales-oriented strategy C9: TOPSIS = 0.50555760
Sales-oriented strategy C10: TOPSIS = 0.98450294
------------------------------
The optimal sales-oriented strategy: C8
```

In [13]:

```python
import networkx as nx
import matplotlib.pyplot as plt

nodes = TOI_nodes  # list các tuple

# ============================================================
# CONFIG
# ============================================================

# Dịch toàn bộ graph sang phải
X_SHIFT = 0.3

# Khoảng cách dọc giữa các level
y_gap = 0.18

# Khoảng cách ngang giữa tâm các node
# 0.002 là quá nhỏ -> gây chồng node
x_gap = 0.16

# Kích thước node
NODE_SIZE = 3000



# ============================================================
# PARENT MAP
# ============================================================

parent_map = {}

for child in nodes:

    if len(child) > 1 and child[:-1] in nodes:
        parent_map[child] = child[:-1]

    else:
        parent_map[child] = None

# ============================================================
# GRAPH
```

```
# ============================================================

G = nx.DiGraph()

# Root
G.add_node("Kosarak")

# Add nodes
for node in nodes:
    G.add_node(node)

# ============================================================
# LABELS
# ============================================================

labels = {}

for node in G.nodes():

    if node == "Kosarak":

        labels[node] = "Kosarak"

    else:

        # Level 1
        if len(node) == 1:

            labels[node] = rf"$\langle${node[-1]}$\rangle$"

        # Level > 1
        else:

            labels[node] = rf"$\langle${node[-1]}$\rangle$"

# ============================================================
# EDGES
# ============================================================

for child, parent in parent_map.items():

    if parent:

        G.add_edge(parent, child)

    else:

        G.add_edge("Kosarak", child)

# ============================================================
# LEVELS
# ============================================================

levels = {
    0: ["Kosarak"]
}
```

```
for node in nodes:

    lvl = len(node)

    levels.setdefault(lvl, []).append(node)

# ============================================================
# NODE POSITION
# ============================================================

pos = {}

for lvl, level_nodes in levels.items():
    n = len(level_nodes)

    # Các node luôn được trải đều từ trái sang phải
    x_positions = [i * x_gap for i in range(n)]

    for i, node in enumerate(level_nodes):
        x = X_SHIFT + x_positions[i]
        y = -lvl * y_gap
        pos[node] = (x, y)

    # ------------------------------------------------------
    # Gán vị trí
    # ------------------------------------------------------

    for i, node in enumerate(level_nodes):

        x = x_positions[i] + X_SHIFT

        y = -lvl * y_gap

        pos[node] = (x, y)

# ============================================================
# DRAW
# ============================================================

fig, ax = plt.subplots(figsize=(7, 9))

nx.draw(
    G,
    pos,
    labels=labels,

    # ------------------------------------------------------
    # Node
    # coral ------------------------------------------------------

    node_size=NODE_SIZE,
    node_color="none",
    edgecolors="black",

    # ------------------------------------------------------
```

```
    # Font
    # ------------------------------------------------------
    font_size=20,
    font_weight="bold",

    # ------------------------------------------------------
    # Edge
    # ------------------------------------------------------
    arrowsize=10,

    ax=ax
)

# ============================================================
# CASE
# ============================================================

fig.text(
    0.5,
    0.94,

    rf"Case $\mathcal{{C}}_{{{index_case}}}$",

    ha="center",
    va="top",

    fontsize=20,
    fontweight="bold"
)

# ============================================================
# RANKING
# ============================================================

fig.text(
    0.18,
    0.91,

    rf"$1st \qquad\quad "
    rf"2nd \qquad\quad "
    rf"3rd \qquad\quad "
    rf"4th \qquad\quad "
    rf"5th$",

    ha="left",
    va="top",

    fontsize=16,
    # fontweight="bold"
)

# ============================================================
# LEVEL LABEL
# ============================================================
```

```
first_pos = 0.705
move_pos = 0.1490
for i in range(1, max_level + 1):
    fig.text(
        0.0,
        first_pos - (i - 1) * move_pos,

        rf"$\ell = {i}$",

        ha="left",
        va="top",

        fontsize=20,
        fontweight="bold"
    )



# ============================================================
# AXIS
# ============================================================

ax.margins(0)

ax.axis("off")

# ============================================================
# AXIS LIMITS
# ============================================================

xs = [x for x, y in pos.values()]
ys = [y for x, y in pos.values()]

# ------------------------------------------------------------
# Tính giới hạn dựa trên node thực tế
# ------------------------------------------------------------

x_min = min(xs)
x_max = max(xs)

y_min = min(ys)
y_max = max(ys)

# Khoảng đệm
x_margin = 0.18
y_margin = 0.08

ax.set_xlim(
    x_min - x_margin,
    x_max + x_margin
)

ax.set_ylim(
    y_min - y_margin,
    y_max + y_margin
)
```

```
# ============================================================
# LAYOUT
# ============================================================

plt.tight_layout(
    rect=[0, 0, 1, 0.94]
)

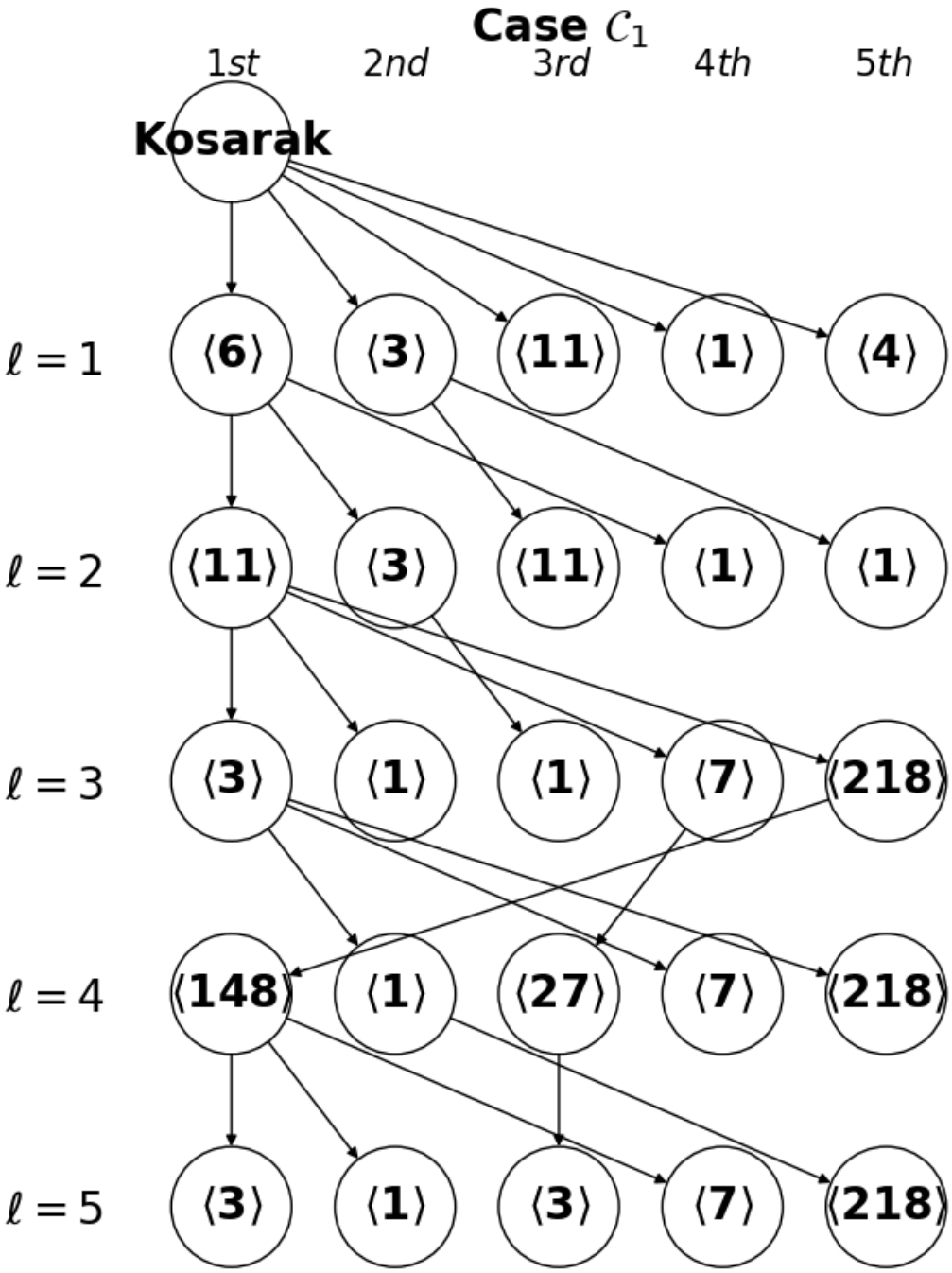
# ============================================================
# SAVE
# ============================================================

plt.savefig(
    f"TOI_Hierarchy_{dbName}_{index_case}.pdf",

    dpi=300,

    bbox_inches="tight"
)

plt.show()
```



In [14]:
```
# import kagglehub

# # Download latest version
# path = kagglehub.dataset_download("roopacalistus/superstore")

# print("Path to dataset files:", path)
```

In [ ]:
```

```